MÉTODOS FORMAIS

Prof. Júlio Machado

INTRODUÇÃO

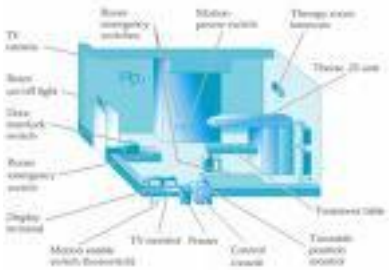
Conceitos Fundamentais

Software e qualidade

- Software é um dos mais variados e complexos produtos produzidos de forma regular.
- Requisitos de qualidade dependentes:
  - Do domínio da aplicação
  - Ambiente de execução
  - Público alvo
  - Etc.
- Exige técnicas sofisticadas e específica para cada produto desenvolvido.

Software e qualidade

- Ariane 5:
  - Explosão em seu primeiro voo.
  - Causado pelo reuso de algumas partes de código de seu predecessor sem verificação adequada.
- Therac-25:
  - Máquina de terapia de radiação.
  - Devido a um erro de software, seis pessoas morreram de overdose.
- Pentium FDIV:
  - Erro de projeto na unidade de divisão de ponto-flutuante.
  - Intel foi forçada a oferecer substituição de todos os processadores defeituosos.



Software e qualidade

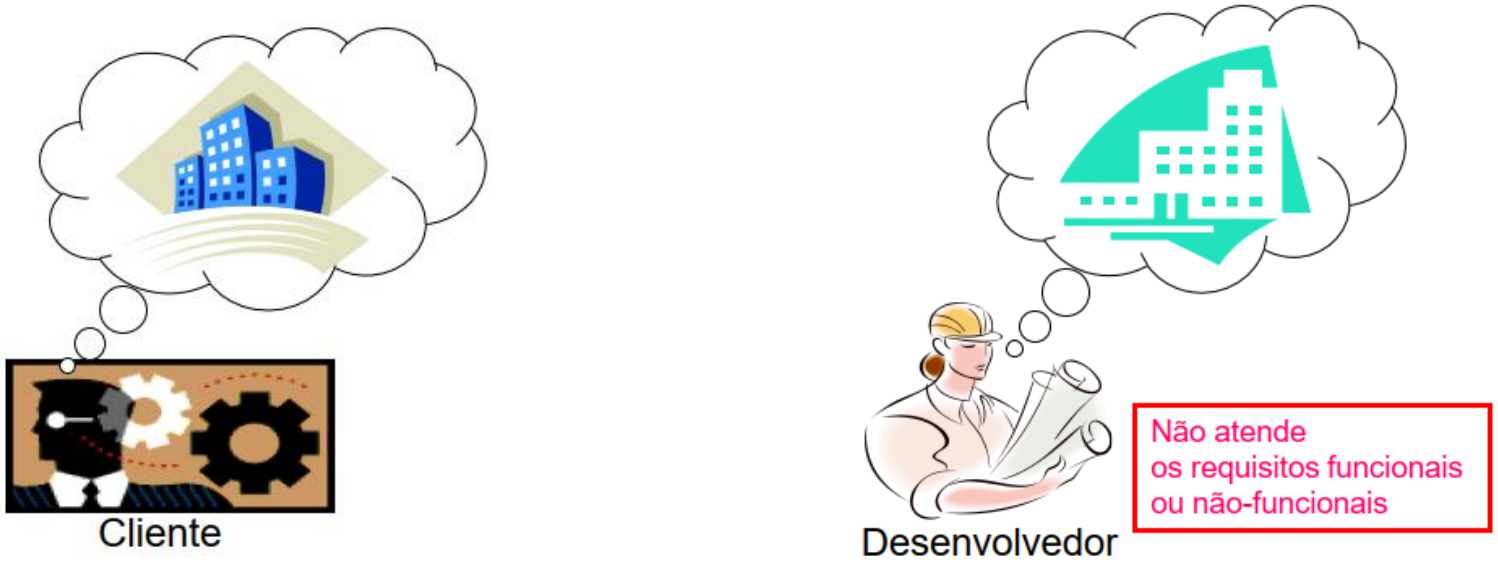
- Por que é tão difícil garantir a qualidade do software?
  - Porque é algo dinâmico.
  - Porque envolve pessoas.
  - Porque as regras de negócio podem ser complexas.
  - Porque as tecnologias mudam rapidamente.
  - Porque as equipes mudam a toda hora.
  - Porque novas necessidades surgem a cada momento.
  - ...

Software e qualidade

- Verificação
- Validação

Verificação

- “Estamos construindo o produto corretamente?”



Verificação

O quê é?

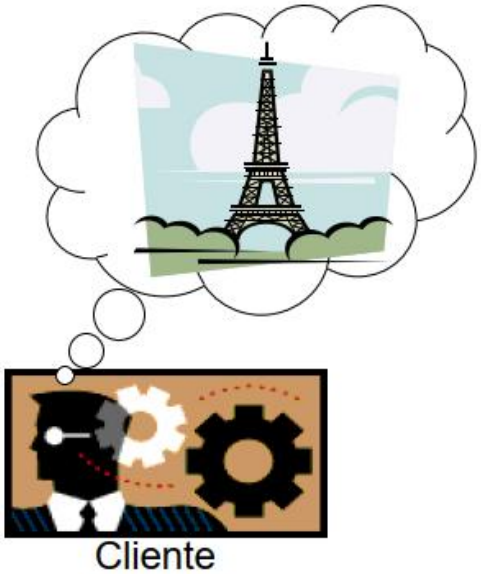
- Processo de garantir que o software está sendo construído corretamente
- Foca na conformidade com os requisitos especificados

Exemplos de técnicas:

- Revisão de Código - Análise estática do código
- Inspeção de Documentação - Revisão de requisitos e especificação
- Análise Estática - Uso de ferramentas para detectar falhas no código sem executá-lo

Validação

- “Construímos o produto correto?”



Validação

O quê é?

- Processo de garantir que o software atende às necessidades do usuário
- Valida se o software está correto para seu uso final

Exemplos de técnicas:

- Testes de Usabilidade - Mede a experiência do usuário
- Testes de Aceitação - Realizados pelo cliente para verificar se o software atende suas necessidades

Validação e Verificação

- **IMPORTANTE**: atividades de verificação e validação apropriadas dependem da disciplina de engenharia, do processo de construção, do produto final e dos requisitos de qualidade.

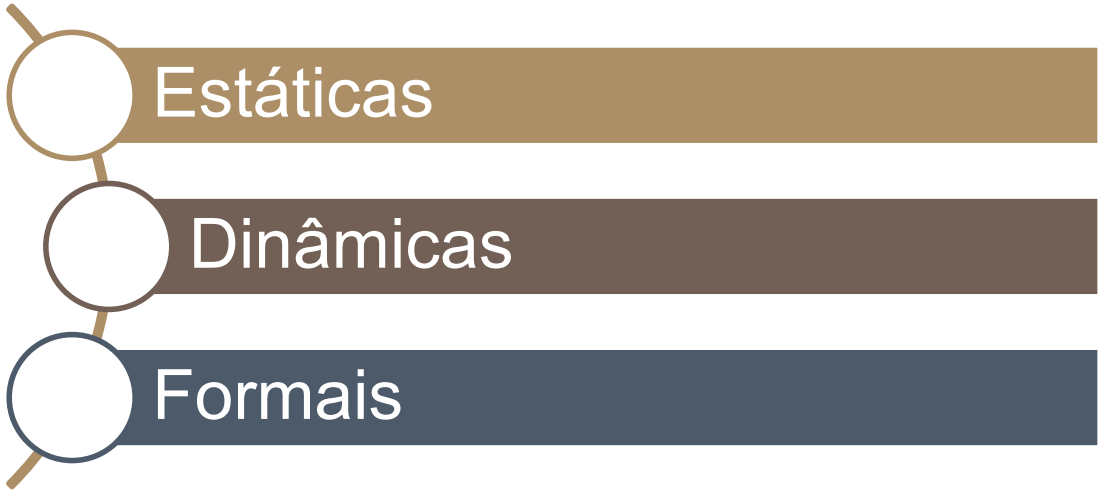
Requisitos

- **Requisitos funcionais** descrevem as funcionalidades que um software deve fornecer.
  - Exemplos: formatar um texto, transmitir uma requisição via *web*.

- **Requisitos não-funcionais** descrevem restrições que atuam sobre um software; também são conhecidos como requisitos de qualidade.
  - Exemplos: o sistema deve transmitir uma requisição via *web* de forma segura e dentro de um prazo limite.

Técnicas de V&V

- Agrupamentos de técnicas de validação e verificação:



Técnicas de V&V

- **Técnicas de V&V estáticas** não requerem que o sistema de software seja executado.
  - Exemplo: revisões manuais de código, análise estática de padrões de código
- **Técnicas de V&V dinâmicas** requerem trabalhar com uma representação executável do sistema de software.
  - Exemplo: teste de software
- **Técnicas de V&V formais** requerem o uso de linguagens formais de especificação e fundamentos matemáticos sólidos.
  - Exemplo: prova de correção

O que é um Método Formal?

- É (a descrição de) qualquer abordagem que utiliza uma linguagem de especificação formal e ferramentas associadas no processo de desenvolvimento de software.
- Entre essas ferramentas, incluem-se:
  - Verificadores de programas
  - Verificadores de modelos
  - Provadores automático de teoremas
  - Assistentes de provas interativos
  - Etc.
- Pode significar o uso somente para documentação ou, preferencialmente, também técnicas de verificação e refinamento com automatização.

O que é uma Linguagem de Especificação Formal?

- Qualquer linguagem na qual é possível especificar completa ou parcialmente a funcionalidade de todo ou parte de um componente de software (ou hardware), e sobre a qual seja possível efetuar algum tipo de raciocínio formal.
- Ela deve possuir uma fundamentação lógico-matemática, em geral na Lógica e na Matemática Discreta.

Lógicas utilizadas em Métodos Formais

- **Lógica de Predicados ou Lógica de Primeira Ordem**
- **Lógicas de Segunda Ordem** (Isabelle, HOL)
- **Lógicas de Programas** (Lógica de Hoare, Lógica Dinâmica, Lógica de Separação)
- **Lógicas Temporais** (Lógica de Tempo Linear, Lógica de Tempo Ramificado)
- **Lógica Equacional** (Especificações Algébricas)

Porque Métodos Formais?

- Forçam os projetistas/desenvolvedores a pensar cuidadosamente sobre a especificação do sistema, evitando o envolvimento precoce com a codificação.
- Mesmo para aqueles profissionais com uma grande experiência em modelagem/programação, as ideias subjacentes aos métodos formais propiciam uma visão completamente nova sobre sistemas de computação em geral.
- Eles reforçam a ideia de uma abordagem de engenharia para o desenvolvimento de sistemas.
- Eles estão gradualmente sendo implantados em projetos industriais.

Quando utilizar Métodos Formais?

- Quando for desejável descrever o comportamento de um software/hardware de forma não ambígua.
- Quando algum tipo de argumentação simbólica sobre a correção de um componente de software/hardware for necessária.
- Quando a utilização de métodos formais é entendida como uma das formas de preencher padrão específico.
- Quando o software/hardware for uma aplicação de segurança crítica ou qualquer outra aplicação onde o custo de falhas no sistema seja muito alto.
- Quando a organização já possui um grupo estabelecido com a experiência necessária para utilizar métodos formais.

VERIFICAÇÃO FORMAL DE SISTEMAS

Introdução

Verificação Formal

- No contexto de sistemas de software e hardware, verificação formal é o ato de **provar** ou **refutar** a correção de um determinado **algoritmo**, **programa** ou **modelo** com relação a uma certa **especificação ou propriedade formal**, utilizando métodos formais.

Abordagens para Verificação Formal

- Verificação de Modelos (*Model Checking*):
  - Consiste em uma exploração sistemática e exaustiva do modelo matemático (usualmente uma máquina de estados ou uma descrição do sistema em uma linguagem lógica que seja decidível).
  - As propriedades a serem verificadas são frequentemente descritas em **lógica temporal**, como lógica temporal linear (LTL) ou lógica temporal ramificada (CTL) ou em alguma variante da lógica de predicados.
  - É baseado em **modelos**.
  - Esta abordagem é suportada por modelos descritos com álgebra de processos ou com uma linguagem subjacente verificadores de modelos (Spin, NuSMV, etc).

Abordagens para Verificação Formal

- Verificação Dedutiva:
  - Consiste na utilização de uma versão formal de raciocínio matemático (lógico) sobre o sistema de forma a estabelecer propriedades desejadas no sistema.
  - As provas são conduzidas dentro de um **Cálculo Lógico.**
  - Normalmente utiliza-se de softwares de apoio como assistentes de prova (Isabelle, Coq, etc) para automatizar certas partes do raciocínio.
  - Verificação baseada em **inferência** (portanto, em **provas**).

Abordagens para Verificação Formal

- Comparação:
  - Na abordagem baseada em provas, a descrição do sistema é um conjunto de fórmulas $Spec$ (em uma determinada lógica) e a especificação é outra fórmula $prop$. O método de verificação consiste em tentar achar uma prova de que $Spec \vdash prop$ ($prop$ é um teorema de $Spec$).
  - Na abordagem baseada em modelos, o sistema é representado por um modelo M para uma lógica apropriada. A especificação é novamente representada por uma fórmula $prop$ e o método de verificação consiste em computar se o modelo M satisfaz $prop$, isto é, se $M \vDash prop$.